PROCESSAMENTO DE ALTO DESEMPENHO

Aula Prática - Programação Paralela com MPI e Python

Decomposição, Comunicação Coletiva e Avaliação de Desempenho

Professor: Márcio Augusto E. de Moraes

1. Objetivos

- Decompor problemas computacionais em tarefas paralelas.
- Distribuir dados entre diferentes processos MPI.
- Utilizar corretamente bcast, scatter, gather, reduce e allreduce.
- Combinar comunicação coletiva e processamento local.
- Tratar situações em que os dados não podem ser divididos igualmente.
- Validar resultados paralelos comparando-os com soluções sequenciais.
- Medir corretamente o tempo de execução.
- Calcular speed-up e eficiência.
- Identificar limitações do paralelismo, custo de comunicação e problemas de balanceamento de carga.

2. Ambiente

Os programas deverão ser executados no ambiente Windows previamente preparado com Python, Microsoft MPI, mpi4py, NumPy e Matplotlib.

```
mpiexec -n 4 python programa.py
```

Bibliotecas básicas:

```
from mpi4py import MPI
import numpy as np
```

3. Regras gerais

1. O programa deverá funcionar com diferentes números de processos.
2. O código não poderá pressupor exclusivamente size = 4.
3. Sempre que possível, apenas o processo rank == 0 deverá apresentar o resultado global.
4. O resultado paralelo deverá ser validado utilizando uma solução sequencial.
5. O tempo de execução deverá ser medido utilizando MPI.Wtime().
6. Quando necessário, utilizar comm.Barrier() antes de iniciar a medição.
7. Os alunos deverão justificar a escolha das operações MPI utilizadas.

4. Problema 1 - Estatísticas distribuídas de um conjunto de dados

Considere um vetor contendo milhões de valores numéricos. O processo 0 deverá criar:

```
N = 10_000_003
dados = np.random.random(N)
```

O valor de N foi escolhido propositalmente para não ser divisível por vários números de processos.

Objetivo

$$S = \Sigma\, xi$$

$$\mu = (1/N)\, \Sigma\, xi$$

Além da soma e da média, determinar o valor máximo e o valor mínimo do conjunto de dados.

Etapa 1 - Distribuição

O processo 0 deverá dividir o vetor em size blocos. Uma possibilidade é:

```
blocos = np.array_split(dados, size)
```

Os blocos deverão ser distribuídos utilizando scatter().

Etapa 2 - Processamento local

Cada processo deverá calcular:

- soma_local
- quantidade_local
- maximo_local
- minimo_local

Etapa 3 - Resultado global

Utilize operações reduce apropriadas para determinar soma_global, quantidade_global, maximo_global e minimo_global. Em seguida, calcule a média global.

Etapa 4 - Validação

No processo 0, compare os resultados com:

```
np.sum(dados)
np.mean(dados)
np.max(dados)
np.min(dados)
```

Questões

8. Por que N não precisa ser divisível pelo número de processos?
9. Por que calcular a média de cada bloco e depois fazer a média das médias pode produzir resultado incorreto?
10. Que informações devem ser reduzidas para calcular corretamente a média global?
11. Qual operador MPI deve ser utilizado para soma, máximo e mínimo?
12. Nesse problema seria necessário utilizar gather()?
13. Qual a vantagem de usar reduce() em vez de reunir todos os resultados no processo 0?

5. Problema 2 - Gather versus Reduce

Utilize novamente o vetor do problema anterior. Cada processo deverá calcular soma_local.

Versão A - Gather

Utilize comm.gather(). O processo 0 receberá uma lista com todas as somas locais e realizará a soma final.

Versão B - Reduce

```
soma_global = comm.reduce(
    soma_local,
    op=MPI.SUM,
    root=0
)
```

Experimento

Execute as duas versões com 1, 2, 4 e 8 processos e registre o tempo de execução.

Questões

14. Os resultados são numericamente equivalentes?
15. Qual solução exige menos código?
16. Em qual solução o processo 0 recebe mais informação?
17. Imagine agora 10.000 processos. Qual abordagem seria conceitualmente mais adequada?
18. Quando gather() seria indispensável?

6. Problema 3 - Integração numérica paralela

Calcular numericamente:

$$I = \int_0^{\pi} \sin(x)\, dx$$

O valor analítico é I = 2.

Utilize a regra do ponto médio:

$$I \approx \Sigma\, f(xi)\, \Delta x$$

$$\Delta x = (b - a)/N$$

$$xi = a + (i + 0{,}5)\Delta x$$

Utilize inicialmente:

```
N = 100_000_000
```

Requisito importante

Neste problema, não distribua um vetor de 100 milhões de valores usando Scatter. Cada processo deverá determinar matematicamente quais índices são de sua responsabilidade.

```
for i in range(rank, N, size):
```

Broadcast

Somente o processo 0 deverá inicialmente definir a, b e N. Esses parâmetros deverão ser distribuídos com bcast().

Processamento

Cada processo calcula integral_local. Depois, reduce(..., MPI.SUM) deverá produzir a integral global.

Questões

19. Por que não foi necessário utilizar scatter()?
20. É melhor transmitir 100 milhões de valores ou transmitir apenas a, b e N?
21. Qual o impacto dessa decisão sobre a comunicação?
22. Qual estratégia de decomposição foi utilizada?
23. Todos os processos recebem aproximadamente a mesma quantidade de trabalho?
24. Qual erro numérico foi obtido em relação ao resultado analítico?

7. Problema 4 - Estimativa de π pelo método de Monte Carlo

Considere um quadrado definido por $-1 \leq x \leq 1$ e $-1 \leq y \leq 1$. Um ponto pertence ao círculo unitário quando:

$$x^2 + y^2 \leq 1$$

A aproximação é:

$$\pi \approx 4\, Ndentro / Ntotal$$

Parte A - Programa sequencial

Implemente inicialmente a versão sequencial utilizando:

```
N = 100_000_000
```

Registre π calculado, erro absoluto e tempo de execução.

Parte B - Programa paralelo

O número total de pontos deverá ser definido no processo 0 e distribuído com bcast(). Cada processo deverá gerar apenas seus próprios pontos aleatórios. Não distribua os pontos pelo processo 0.

Cada processo calcula:

- pontos_dentro_local
- pontos_gerados_local

Depois utilize operações de redução para obter os totais globais.

Atenção às sementes aleatórias

Cada processo deverá utilizar uma sequência aleatória diferente. Uma estratégia simples é:

```
seed = seed_base + rank
```

Questões

25. O que aconteceria se todos os processos utilizassem exatamente a mesma sequência de números aleatórios?
26. É necessário usar scatter() neste problema?
27. Por que gerar os dados localmente pode ser melhor?
28. O erro da estimativa diminui necessariamente quando aumentamos o número de processos?
29. Aumentar o número de processos aumenta a precisão ou apenas reduz o tempo para executar a mesma quantidade de experimentos?

8. Problema 5 - Multiplicação matriz-vetor paralela

Considere:

$$y = A\,x$$

Utilize inicialmente:

```
N = 8000
A = np.random.random((N, N))
x = np.random.random(N)
```

Atenção à quantidade de memória disponível no laboratório. Se necessário, reduza N.

Estratégia

30. O processo 0 possui inicialmente A e x.
31. O vetor x é necessário para todos os processos. Escolha a operação coletiva apropriada.
32. A matriz A deverá ser dividida por linhas e distribuída entre os processos.
33. Cada processo calcula y_local = A_local @ x.
34. Os diferentes pedaços de y deverão ser reunidos no processo 0.

Fluxo conceitual:

```
Processo 0: A, x
 x -> Broadcast
 A -> Scatter por linhas
 Processos -> cálculo local A_local @ x
 resultados -> Gather
 Processo 0 -> vetor y
```

Validação

```
y_numpy = A @ x
np.allclose(y_paralelo, y_numpy)
```

Questões

35. Por que x é transmitido usando Broadcast?
36. Por que A é distribuída usando Scatter?
37. Por que o resultado utiliza Gather e não Reduce?
38. Seria possível obter y usando apenas uma soma global?
39. Qual quantidade de dados cresce mais rapidamente quando N aumenta: A ou x?

9. Problema 6 - Contagem paralela de números primos

Determinar quantos números primos existem no intervalo:

$$2 \leq n \leq N$$

```
N = 2_000_000
```

O processo 0 não deverá criar um vetor contendo todos os números e distribuí-lo. Cada processo deverá determinar seu próprio intervalo ou conjunto de índices.

Parte A - Decomposição em blocos

Cada processo recebe uma faixa contínua do intervalo.

Parte B - Decomposição cíclica

```
P0 -> 2, 6, 10, 14, ...
P1 -> 3, 7, 11, 15, ...
P2 -> 4, 8, 12, 16, ...
P3 -> 5, 9, 13, 17, ...
```

Compare os tempos das duas estratégias.

Questões

40. Qual estratégia apresentou melhor balanceamento?
41. Todos os números possuem o mesmo custo para determinar se são primos?
42. Como a irregularidade do custo computacional pode afetar o balanceamento?
43. Qual é a relação entre balanceamento de carga e eficiência?

10. Problema 7 - Medição correta do tempo paralelo

Cada processo deverá medir seu próprio tempo:

```
inicio = MPI.Wtime()
# processamento
fim = MPI.Wtime()
tempo_local = fim - inicio
```

Depois utilize MPI.MAX em uma redução para determinar o maior tempo entre os processos. Esse valor será utilizado como tempo da aplicação paralela.

Questões

44. Por que utilizar o maior tempo?
45. O que significa um processo terminar muito antes dos outros?
46. Isso pode indicar problema de balanceamento?

11. Avaliação de desempenho

Escolha pelo menos dois problemas computacionalmente intensivos desta prática. Recomenda-se integração numérica, Monte Carlo e números primos.

Execute usando 1, 2, 4 e 8 processos. Se o computador permitir, teste também 12 e 16 processos.

Processos	T(p)	Speed-up	Eficiência
1		1,00	100%
2			
4			
8			
16			

$$S(p) = T(1) / T(p)$$

$$E(p) = S(p) / p$$

12. Gráficos

- Gráfico de speed-up: número de processos no eixo X e speed-up no eixo Y. Inclua também a reta ideal S(p) = p.
- Gráfico de eficiência: número de processos no eixo X e eficiência (%) no eixo Y.

13. Análise obrigatória

Não basta afirmar que 'com mais processos ficou mais rápido'. Discuta:

- custo computacional;
- custo de comunicação;
- criação e gerenciamento de processos;
- sincronização;
- balanceamento de carga;
- tamanho do problema;
- número de núcleos físicos;
- número de processadores lógicos;
- possível oversubscription.

14. Experimento - Quando paralelizar não vale a pena?

Escolha um dos algoritmos e execute com diferentes tamanhos do problema, por exemplo:

N = 1_000
N = 10_000
N = 100_000
N = 1_000_000
N = 10_000_000

Para cada N, utilize 1, 2, 4 e 8 processos. Determine aproximadamente a partir de qual tamanho o paralelismo começa a trazer vantagem.

15. Desafio avançado - Equação do calor 1D

Considere uma barra discretizada em N pontos. A evolução da temperatura pode ser calculada aproximadamente por:

$$Ti^{(n+1)} = Ti^n + \alpha\,(Ti\text{-}1^n - 2Ti^n + Ti\text{+}1^n)$$

Cada processo será responsável por uma parte da barra. Para atualizar os pontos próximos às fronteiras, cada processo precisa conhecer valores pertencentes ao processo vizinho.

Portanto, será necessário combinar comunicação coletiva com comunicação ponto a ponto.

```
comm.send()
comm.recv()
```

Ou, opcionalmente, comunicação não bloqueante.

Critério de convergência

$$erro_local = max\ |Ti^{(n+1)} - Ti^n|$$

Utilize:

```
erro_global = comm.allreduce(
   erro_local,
   op=MPI.MAX
)
```

Todos os processos deverão interromper a simulação quando erro_global < ε.

16. Por que Allreduce?

No reduce(), somente o processo root recebe o resultado. No problema da equação do calor, todos os processos precisam saber se a simulação convergiu. Por isso, allreduce() é mais apropriado.

17. Desafio opcional - Scatterv e Gatherv

Para grandes arrays NumPy, o mpi4py também oferece operações diretamente sobre buffers:

```
Bcast()
Scatter()
Gather()
Reduce()
Scatterv()
Gatherv()
```

Reimplemente o Problema 1 utilizando arrays NumPy, Scatterv e Reduce. Compare o desempenho com a versão baseada em objetos Python.

18. Desafio final - Escolha a estratégia MPI

Situação	Descrição	Operação MPI
A	O processo 0 lê uma constante	_____

	que todos deverão utilizar.	
B	Uma matriz deverá ser dividida entre os processos.	_____
C	Cada processo encontrou um valor e precisamos de todos os valores.	_____
D	Cada processo calculou uma soma parcial e precisamos apenas da soma global.	_____
E	Todos os processos precisam conhecer a soma global.	_____

19. Mini-projeto

Escolha uma das seguintes opções:

- Opção A - Mandelbrot paralelo: dividir as linhas da imagem entre processos e reunir o resultado com Gather.
- Opção B - Monte Carlo tridimensional: estimar o volume de uma esfera utilizando milhões de pontos aleatórios.
- Opção C - Multiplicação de matrizes: paralelizar parte do cálculo C = AB e avaliar desempenho.
- Opção D - Processamento de série temporal: dividir os dados entre processos e calcular média, máximo, mínimo, desvio padrão e quantidade de valores acima de um limiar.

20. Arquivos para entrega

```
GrupoXX_MPI/
|
|-- problema1_estatisticas.py
|-- problema2_gather_reduce.py
|-- problema3_integracao.py
|-- problema4_montecarlo.py
|-- problema5_matriz_vetor.py
|-- problema6_primos.py
|
|-- resultados.csv
|-- speedup.png
|-- eficiencia.png
|
`-- relatorio.pdf
```

21. Relatório

47. Estratégia: como cada problema foi decomposto.
48. Comunicação: quais operações MPI foram utilizadas e por quê.

49. Validação: comparação entre resultado sequencial e paralelo.
50. Desempenho: tempos para diferentes números de processos.
51. Speed-up: análise do ganho obtido.
52. Eficiência: análise do aproveitamento dos processos.
53. Limitações: fatores que impediram speed-up ideal.
54. Conclusão: avaliar quando o uso de MPI foi realmente vantajoso.

22. Critérios de avaliação

Critério	Peso
Correção dos algoritmos	20%
Decomposição do problema	20%
Uso correto do MPI	20%
Validação dos resultados	10%
Avaliação de desempenho	15%
Discussão científica/técnica	10%
Organização e qualidade do código	5%

23. Questões finais

55. Qual a diferença entre bcast e scatter?
56. Qual a diferença entre gather e reduce?
57. Quando allreduce é mais adequado que reduce?
58. Por que nem todo problema precisa utilizar Scatter?
59. Em quais situações é melhor gerar os dados diretamente em cada processo?
60. Qual a influência do volume de comunicação no speed-up?
61. Por que o tempo do processo mais lento deve ser utilizado como tempo da aplicação?
62. O que caracteriza um problema mal balanceado?
63. É possível obter speed-up menor que 1? O que isso significa?
64. É possível utilizar 16 processos em uma máquina com 8 núcleos? Isso necessariamente melhorará o desempenho?
65. Qual a diferença entre aumentar o número de processos e aumentar o tamanho do problema?
66. Qual das aplicações estudadas apresentou melhor escalabilidade?
67. Para o hardware utilizado no laboratório, qual parece ser o número mais adequado de processos?

24. Objetivo conceitual final

A principal questão desta aula não é apenas como utilizar Scatter, Gather, Broadcast e Reduce.

A questão central é: dado um problema computacional, como decidir quais dados devem ser distribuídos, quais informações precisam ser compartilhadas, quais resultados precisam ser reunidos e quais devem apenas ser reduzidos?